

“Dairy & Milk Cooperative Management System”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

By

T. JASWANTH REDDY	2520030284
S. PREM SAI	2520030561
V. NITISH CHOWDHARY	2520030179

Under the Esteemed Guidance of

Dr. Y Prasanthi Assistant Professor
Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



Declaration

The Project Report entitled **“Dairy & Milk Cooperative Management System”** is a record of Bonafide work of **T. JASWANTH REDDY - 2520030284, S. PREM SAI - 2520030561, V. NITISH CHOWDHARY - 2520030179** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering (or) Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

T. JASWANTH REDDY	2520030284
S. PREM SAI	2520030561
V. NITISH CHOWDHARY	2520030179

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



CERTIFICATE

This is certify that the mini project based report entitled **“Dairy & Milk Cooperative Management System”** is a bonafide work done and submitted by **T. Jaswanth Reddy – 2520030284, S. Prem Sai – 2520030561 and V. Nitish Chowdhary – 2520030179**

in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year **2026-2027**.

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my Course Coordinator [Course Coordinator Name], and my guide Dr. Y Prasanthi, Department of Computer Science which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to Dr. P Venkateshwara Rao (CSE), Head of the *Department for Computer Science Engineering* for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

T. JASWANTH REDDY 2520030284

S. PREM SAI 2520030561

V. NITISH CHOWDHARY 2520030179

Abstract

Dairy cooperatives distribute milk and milk products through a network of small merchant outlets, yet most of these outlets still record stock, sales and payments in paper registers. As a result, stock drifts away from recorded sales, cash and UPI collections cannot be matched to individual bills, and cooperative management learns about revenue and shortages only after slow manual reconciliation. This project, the Dairy & Milk Cooperative Management System, is a web-based platform that digitises the merchant side of a cooperative so that every stock entry, sale and payment is recorded consistently and audited automatically.

The system provides role-based dashboards for cooperative administrators and owners and for outlet merchants. Merchants record incoming stock, bill customers through a multi-item point-of-sale screen and accept cash or UPI payments through the Cashfree payment gateway, while administrators manage merchants, the product catalogue and quality records (fat %, SNF % and grade) and monitor revenue across all outlets. The frontend is built with React and Vite, and the backend consists of seven Node.js/Express microservices behind an API gateway, backed by a normalised MySQL database of ten core tables, indexes and three analytical views.

The key contribution of the project is a consistent sale workflow across distributed services. Each sale is executed as an orchestrated saga: stock is reserved under row-level locks with a time-limited reservation, the payment is verified, and the invoice, sale, line items and payment are written in a single ACID transaction, with compensating actions such as stock release and refunds when a step fails. SQL views continuously compare incoming, sold and available stock and billed versus collected amounts, flagging stock and payment discrepancies for review. The result is a low-cost system that prevents overselling, protects pricing from client-side tampering and gives the cooperative real-time visibility into its operations.

Index

S. No.	Chapter	Topics / Subtopics	Page No.
	Acknowledgement		i
	Abstract		ii
1	Introduction		1–4
		1.1 Background of the Project	1
		1.2 Problem Statement	1
		1.3 Scope of the Project	2
		1.4 Objectives of the Project	3
		1.5 Importance of the Application	3
		1.6 Target Users / Audience	3
2	Database Concepts and Background Study		5–8
		2.1 Database and Distributed Backend Concepts Related to the Project	5
		2.2 Existing Methods / Systems	6
		2.3 Proposed Approach	7
		2.4 Advantages of the Proposed Approach	8
3	System Requirements		9–10
		3.1 Hardware Requirements	9
		3.2 Software Requirements	9
		3.3 Operating System Environment – Windows / Ubuntu	9
		3.4 Development Tools and Runtimes	10
4	Technology And Tools Used		11–13
		4.1 React.js and Vite (Front-End)	11
		4.2 Node.js and Express.js (Back-End)	11
		4.3 MySQL and SQL (Database)	11
		4.4 JWT Authentication and bcrypt	12
		4.5 Cashfree Payment Gateway	12
		4.6 Other Tools / Libraries Used	12
5	System Architecture And Design		14–21

		5.1 Overall System Architecture	14
		5.2 System Workflow	15
		5.3 Flowchart / Data Flow	16
		5.4 Module Description	18
		5.5 Input and Output Design	19
		5.6 Database Design	20
6	Implementation		22–31
		6.1 Module-wise Implementation	22
		6.2 Algorithm / Pseudocode	23
		6.3 Core Back-End Implementation (Node.js / Express)	24
		6.4 REST APIs Used	26
		6.5 Microservices Implementation	27
		6.6 Transactions and Concurrency Control (Saga Implementation)	27
		6.7 Database Implementation (Views, Indexes and Constraints)	29
		6.8 User Interface Implementation	31
7	Execution And Demonstration		32–43
		7.1 Installation and Database Setup	32
		7.2 Execution Commands	33
		7.3 Sample Inputs and Outputs	33
		7.4 Screenshots of Execution	34
		7.5 Demonstration of Database Concepts	42
8	Conclusion		44–45
		8.1 Summary of the Project	44
		8.2 Objectives Achieved	44
		8.3 Results Achieved	44
		8.4 Skills and Learning Outcomes	45
9	References		46–47
		9.1 Books	46
		9.2 Research Papers	46
		9.3 MySQL / Node.js Documentation	46
		9.4 Online Resources	46

CHAPTER -1 INTRODUCTION

1.1 Background of the Project

Dairy cooperatives play a central role in India's milk economy. A typical cooperative collects milk from member farmers, processes it into products such as packaged milk, curd, paneer, ghee and buttermilk, and distributes these products through a network of small retail outlets called parlours or milk centres. Each outlet is run by a merchant who receives stock from the cooperative, sells it to walk-in customers across a morning or evening shift, and settles the collected money with the cooperative.

In most mandal- and town-level cooperatives this distribution chain is still managed on paper. Incoming stock is written in a register, sales are billed by hand or not billed at all, and cash is counted at the end of the day. When the totals do not match, there is no reliable record that shows whether the difference came from spoilage, an arithmetic mistake, an unrecorded sale or deliberate leakage. The cooperative's management therefore learns about revenue and stock positions only after a slow, manual reconciliation.

The growth of digital payments in India, especially UPI, has made it practical for even small outlets to accept online payments. At the same time, modern web technologies make it possible to build a low-cost system that a merchant can run from a phone or a basic computer. This project, the **Dairy & Milk Cooperative Management System**, applies these technologies to the merchant side of the cooperative: it digitises products, stock, point-of-sale billing, payments and invoices, and continuously compares what was received, sold and paid so that discrepancies are surfaced automatically.

The project was developed as part of the course **Database Systems Engineering and Distributed Backend Development (25CS1302E)**. Accordingly, its design emphasises a normalised relational schema in MySQL, ACID transactions, SQL views for analytics, and a backend that evolved from a single Express application into a set of independently running microservices coordinated through the Saga pattern.

1.2 Problem Statement

Dairy parlours that operate under a cooperative face the following problems:

- **No reliable stock record:** stock received from the cooperative is noted manually, so the quantity that should be on the shelf cannot be compared with the quantity actually

present.

- **Unbilled and unverifiable sales:** handwritten bills are easy to skip or alter, and multi-item orders are prone to calculation errors.
- **Untracked payments:** cash and UPI receipts are not linked to individual sales, so a sale that was never paid for is indistinguishable from one that was.
- **Delayed visibility for management:** the cooperative sees revenue and stock only after manual end-of-day or end-of-week reconciliation.
- **Leakage goes unnoticed:** without automated comparison of incoming stock, sold quantity and collected money, shortages are discovered late or never.

The problem addressed by this project is therefore: **to design and develop a database-backed, multi-user web system that records every stock entry, sale and payment of a dairy cooperative's merchant outlets consistently, and automatically detects stock and payment discrepancies.**

1.3 Scope of the Project

The scope of the implemented system covers the merchant distribution side of a dairy cooperative:

- **Users and roles:** three roles are supported: Admin and Owner (cooperative management) and Merchant (outlet operator). Each role sees a different dashboard and set of operations.
- **Modules:** authentication, merchant management, product catalogue with quality records (fat %, SNF %, grade), stock inflow, inventory, point-of-sale billing, payments (cash and Cashfree UPI), invoices, and discrepancy reporting.
- **Data:** all operational data is stored in a single MySQL database (dairy_merchant_db) consisting of ten core tables, two supporting tables used for distributed transactions, and three reporting views.
- **Boundaries:** farmer-side milk collection, fat/SNF-based procurement pricing and farmer payment statements, which were part of the initial abstract, are not implemented in this version and are listed as future scope. Payment integration runs against the Cashfree sandbox environment.

1.4 Objectives of the Project

1. To design a normalised relational database that models merchants, products, product quality, stock entries, inventory, sales, sale items, invoices and payments with proper keys and integrity constraints.
2. To record multi-item sales atomically, so that the invoice, sale, line items and payment are either all saved or none are saved.
3. To prevent overselling by reserving stock with row-level locks before payment is taken.
4. To accept both cash and online UPI payments through the Cashfree payment gateway, with prices always taken from the server rather than the client.
5. To automatically detect stock discrepancies (expected stock versus actual stock) and payment discrepancies (billed amount versus collected amount) using SQL views.
6. To provide role-based dashboards that give the cooperative real-time visibility into revenue, inventory and flagged anomalies.
7. To structure the backend as independent services behind an API gateway, demonstrating distributed backend concepts such as service decomposition and the Saga pattern.

1.5 Importance of the Application

The application matters to each party in the cooperative chain. For **merchants**, it replaces handwritten billing with a fast point-of-sale screen that computes totals, generates an invoice number and records the payment mode in one step. For the **cooperative management**, it turns the end-of-day reconciliation into a live dashboard: revenue billed, revenue collected and flagged discrepancies are visible at any time. For **customers**, it provides an itemised invoice and the option to pay through any UPI app.

From a technical perspective, the project shows how core database concepts are applied in a working product. Transactions protect financial consistency, row locks prevent two simultaneous sales from selling the same stock twice, views turn raw records into audit reports, and the Saga pattern extends consistency across services that cannot share a single transaction.

1.6 Target Users / Audience

Table 1.1: Target users and their primary activities

User	Role in system	Primary activities
------	----------------	--------------------

User	Role in system	Primary activities
Cooperative Admin	ADMIN	Manage merchants and products, record quality tests, view all sales, payments and inventory, review discrepancies
Cooperative Owner	OWNER	Same management view as Admin; monitor revenue and fraud indicators across outlets
Outlet Merchant	MERCHANT	Record incoming stock, bill customers at the POS, accept cash or UPI, view own inventory and payment history
Walk-in Customer	(no login)	Receives an invoice and pays by cash or UPI at the counter

The secondary audience of this report includes faculty evaluators and future developers who may extend the system, for whom the design and implementation chapters document the schema, APIs and transaction logic in detail.

CHAPTER -2 DATABASE CONCEPTS AND BACKGROUND STUDY

2.1 Database and Distributed Backend Concepts Related to the Project

This section summarises the concepts from database systems and distributed backend development that the project directly applies.

Relational model and normalisation

Data is organised into relations (tables) with primary keys and foreign keys, following Codd's relational model. The schema is normalised to Third Normal Form: product details are stored once in products and referenced by sale_items, stock_entries and inventory; merchant details are stored once in merchants and referenced by sales and invoices; and the line items of a sale are separated from the sale header. This avoids update anomalies such as a product price being changed in one place but not another.

Integrity constraints

Entity integrity is enforced through AUTO_INCREMENT primary keys, and referential integrity through named FOREIGN KEY constraints. Domain integrity is enforced through ENUM types (for roles, shifts, statuses and payment methods), CHECK constraints (for example, quantity > 0 and total_amount >= 0) and UNIQUE constraints (for email, merchant code, invoice number and the merchant-product pair in inventory).

Transactions and ACID properties

A sale touches four tables: invoices, sales, sale_items and payments. These inserts run inside a single InnoDB transaction (BEGIN ... COMMIT). If any statement fails, the transaction is rolled back, guaranteeing atomicity and consistency. Durability is provided by InnoDB's redo log, and isolation by its default REPEATABLE READ level.

Concurrency control and row-level locking

Two merchants' terminals, or two quick clicks on the same terminal, could otherwise sell the same last unit of stock. The inventory service prevents this with SELECT ... FOR UPDATE, which places an exclusive lock on the inventory row until the reserving transaction commits. A second transaction trying to reserve the same row waits, and then sees the reduced quantity.

Views, joins and aggregation

Reporting is implemented with three SQL views. They use LEFT JOINs, GROUP BY and SUM aggregates, and COALESCE for missing values. Aggregation is done in derived tables before joining, which avoids the well-known fan-out problem where joining two one-to-many

relationships multiplies rows and inflates totals.

Indexing

Four secondary indexes support the most frequent lookups: `sales(merchant_id, sale_date)` for a merchant's sales history, `stock_entries(merchant_id, product_id)` for per-product stock totals, `sale_items(product_id)` for sold quantities, and `payments(payment_status)` for filtering successful or failed payments. InnoDB also indexes every foreign key column automatically.

Microservices and the API Gateway pattern

A microservice architecture splits a backend into small services, each responsible for one business capability and deployable on its own. Clients call a single API gateway, which routes each request to the correct service. This project uses seven Node.js services (auth, merchant, product, inventory, sales, payment and reporting) behind a gateway on port 4000.

Distributed transactions and the Saga pattern

Once stock, payments and sales live in different services, a single database transaction can no longer span the whole sale. The Saga pattern, introduced by Garcia-Molina and Salem (1987), replaces one long transaction with a sequence of local transactions, each with a compensating action. If a later step fails, earlier steps are undone by their compensations, for example releasing reserved stock or refunding a payment. The sales service acts as the saga orchestrator and records progress in a `saga_instances` table so that interrupted sagas can be recovered.

Authentication and authorisation

Passwords are stored as bcrypt hashes. After a successful login the auth service issues a JSON Web Token (JWT, RFC 7519) containing the user id, role and merchant id. Every service verifies this token and applies role-based access control, for example only ADMIN and OWNER can read discrepancy reports.

2.2 Existing Methods / Systems

Three approaches are commonly used by small dairy outlets today:

- **Manual registers:** stock, sales and cash are written in notebooks. This is cheap but slow, error-prone and impossible to audit in real time.
- **Spreadsheets:** some cooperatives maintain Excel sheets updated at the end of the day. These improve arithmetic but still depend on manual entry, have no access control, and offer no transactional guarantees when several people edit the same file.
- **Generic POS or billing apps:** off-the-shelf billing apps record sales but are not

connected to the cooperative's stock supply, have no notion of expected versus actual stock, and do not reconcile payments against sales across outlets.

Table 2.1: Comparison of the existing and proposed systems

Feature	Existing approach	Proposed system
Stock tracking	Manual register, updated irregularly	Inventory updated automatically on every stock entry and sale
Billing	Handwritten bills, frequent arithmetic errors	Multi-item POS with server-side pricing and unique invoice numbers
Payments	Mostly cash, not linked to sales	Cash and Cashfree UPI, each payment linked to its sale
Concurrency	Not applicable / overselling possible	Row-level locks and reservations prevent overselling
Revenue visibility	End-of-day manual tally	Live admin dashboard
Fraud detection	Discovered late or never	SQL views flag stock and payment discrepancies automatically
Access control	None	JWT-based login with Admin, Owner and Merchant roles

2.3 Proposed Approach

The proposed system is a three-tier web application. A React single-page application provides separate dashboards for administrators and merchants. All requests go through an API gateway to seven Node.js/Express microservices. The services share one MySQL database, in which each service owns a clear set of tables.

The project followed an incremental migration. It was first built as a single Express backend (the monolith) with controllers for each module. The microservices were then extracted one by one using the strangler approach: the gateway initially forwarded unmigrated routes to the monolith, and once every route was served by a dedicated service, the frontend was switched to the gateway and the monolith route was retired.

The central workflow, recording a sale, is implemented as an orchestrated saga:

1. The sales service validates the order and fetches the official price of each product from the product service.
2. The inventory service reserves the requested quantities using row locks; the reservation expires automatically after five minutes if it is not committed.
3. For UPI payments the payment service verifies the Cashfree order; cash payments are

accepted at the counter.

4. The sales service writes the invoice, sale, line items and payment in one ACID transaction.
5. The inventory reservation is committed. If any step fails, previous steps are compensated.

2.4 Advantages of the Proposed Approach

- **Financial consistency:** a sale can never be saved without its line items and payment record, because all four inserts share one transaction.
- **No overselling:** stock is checked and reserved under an exclusive row lock before payment is taken.
- **Tamper-resistant pricing:** prices are always read from the database, so a modified client cannot bill at a lower price.
- **Automatic audit trail:** stock and payment discrepancies are computed continuously by SQL views rather than by manual reconciliation.
- **Fault tolerance:** the saga log allows interrupted sales to be recovered, and payments without a completed sale are detected as orphans.
- **Modularity:** each service can be developed, tested and restarted independently, and the gateway gives the frontend a single stable entry point.
- **Low cost:** the stack is entirely open source, and Cashfree's sandbox allows payment flows to be developed and demonstrated without charges.

CHAPTER -3 SYSTEM REQUIREMENTS

3.1 Hardware Requirements

Table 3.1: Hardware requirements

Component	Minimum (development)	Recommended
Processor	Dual-core 64-bit CPU (Intel i3 / AMD Ryzen 3)	Quad-core (Intel i5 / Ryzen 5) or better
RAM	4 GB	8 GB or more (eight Node.js processes, MySQL and the browser run together)
Storage	2 GB free disk space	SSD with 5 GB free
Network	Internet access for npm install and Cashfree sandbox	Stable broadband
Client device	Any device with a modern browser	Desktop or tablet for the POS screen

3.2 Software Requirements

Table 3.2: Software requirements

Software	Version used	Purpose
Node.js	v18 or later (v22 tested)	Runtime for the gateway and all microservices
npm	v9 or later	Package management
MySQL Server	8.0	Relational database (InnoDB engine)
React	18.3	Frontend user interface library
Vite	5.3	Frontend development server and build tool
Express	4.19 – 4.21	HTTP framework for each service
mysql2	3.10 – 3.12	MySQL driver with promise and connection-pool support
jsonwebtoken	9.0	JWT creation and verification
bcryptjs	2.4	Password hashing
Web browser	Chrome, Edge or Firefox (latest)	Running the application

3.3 Operating System Environment – Windows / Ubuntu

The system is platform-independent because every component runs on Node.js and MySQL. It was developed on Windows 10/11, and it runs unchanged on Ubuntu 22.04 or later and on Windows Subsystem for Linux (WSL 2). On Ubuntu, Node.js can be installed from the NodeSource repository and MySQL with apt install mysql-server. Ports 4000 to 4007 must be free for the gateway and services, and port 5173 for the Vite development server.

3.4 Development Tools and Runtimes

Table 3.3: Development tools

Tool	Use in the project
Visual Studio Code	Source code editing for frontend and backend
MySQL Workbench / mysql CLI	Running schema.sql, inspecting tables and views
Git and GitHub	Version control and team collaboration
Postman	Manual testing of REST endpoints
Draw.io	Entity-relationship diagram
Chrome DevTools	Debugging the React frontend and network calls
Node.js test scripts	Integration test suites for the saga, gateway and reporting services

CHAPTER -4 TECHNOLOGY AND TOOLS USED

4.1 React.js and Vite (Front-End)

React is a JavaScript library for building user interfaces from reusable components. The frontend is a single-page application written in React 18 and bundled with Vite 5, which provides a fast development server with hot module replacement and an optimised production build. React Router 6 handles client-side routing, and the routes are wrapped in a ProtectedRoute component that only renders a page when the logged-in user has the required role.

The application has two route groups. The /admin pages (Dashboard, Merchants, Products Catalog, Sales Records, Payment Ledger, Global Inventory and Discrepancies) are available to Admin and Owner users. The /merchant pages (Overview, Products Price List, Record Stock In, My Inventory, Point of Sale and Payment History) are available to merchants. Authentication state is kept in a React context (AuthContext), which stores the JWT and user profile in the browser and refreshes the profile from /api/auth/me on load.

All HTTP calls go through a shared Axios instance whose request interceptor attaches the JWT as a Bearer token. During development, Vite proxies every /api request to the API gateway on port 4000, so the browser only ever talks to a single origin.

4.2 Node.js and Express.js (Back-End)

Node.js is a JavaScript runtime with an event-driven, non-blocking I/O model, which suits APIs that spend most of their time waiting on the database or on other services. Express is a minimal web framework for Node.js that provides routing and middleware. Each of the eight backend processes (the gateway and seven services) is a separate Express application with its own package.json, port and .env configuration.

Common middleware in every service includes cors for cross-origin requests, express.json for parsing request bodies, an authenticateToken function that verifies the JWT, and an authorizeRole function that restricts an endpoint to particular roles. A root script, start_all.js, launches the gateway and all services together for development.

4.3 MySQL and SQL (Database)

MySQL 8.0 with the InnoDB storage engine is used as the relational database. InnoDB was chosen because it supports transactions, foreign keys, row-level locking and CHECK constraints, all of which the system depends on. The entire schema, including tables, indexes, views and seed

data, is defined in database/schema.sql and can be recreated with a single command.

The services connect through the mysql2 driver using connection pools. Every query uses parameter placeholders (?), so user input is never concatenated into SQL, which protects against SQL injection. Multi-statement operations obtain a dedicated connection from the pool, call beginTransaction, and finish with commit or rollback.

4.4 JWT Authentication and bcrypt

User passwords are never stored in plain text; the users table stores a bcrypt hash produced with a cost factor of 10. At login, the auth service accepts an email address, merchant code or merchant id together with the password, compares the password using bcrypt.compare, checks that the account is ACTIVE, and issues a signed JWT.

The token payload carries the user_id, role and name; services that need the merchant look up merchant_id from the user_id. Because every service verifies the same token, no service needs to call the auth service on every request, which keeps the services loosely coupled. Role checks are applied per endpoint: for example, creating products is limited to ADMIN, and reading the discrepancy reports to ADMIN and OWNER.

4.5 Cashfree Payment Gateway

Cashfree Payments is an Indian payment gateway that supports UPI, cards and net banking. The payment service integrates with Cashfree's Orders API: it creates an order for the bill amount, returns a payment session to the frontend, which opens Cashfree's checkout, and afterwards verifies the order status before the sale is saved. A refund endpoint is used by the saga as the compensating action when a payment succeeded but the sale could not be recorded. Development and demonstration use Cashfree's sandbox (TEST) environment, so no real money is moved.

4.6 Other Tools / Libraries Used

Table 4.1: Libraries used in the project

Library	Layer	Purpose
react-router-dom 6	Frontend	Client-side routing and protected routes
axios 1.7	Frontend	HTTP client with JWT interceptor
express 4	Backend	HTTP server and routing in each service
mysql2 3	Backend	MySQL driver with promise-based connection pools
jsonwebtoken 9	Backend	Signing and verifying JWTs

Library	Layer	Purpose
bcryptjs 2.4	Backend	Password hashing
cors, dotenv	Backend	Cross-origin requests and environment configuration
concurrently / start_all.js	Tooling	Starting all services together
Node fetch API	Backend	Service-to-service HTTP calls and gateway forwarding

CHAPTER -5 SYSTEM ARCHITECTURE AND DESIGN

5.1 Overall System Architecture

The system follows a layered, service-oriented architecture with four tiers: the React client, the API gateway, the microservices, and the MySQL database. Figure 5.1 shows the components and how they communicate.

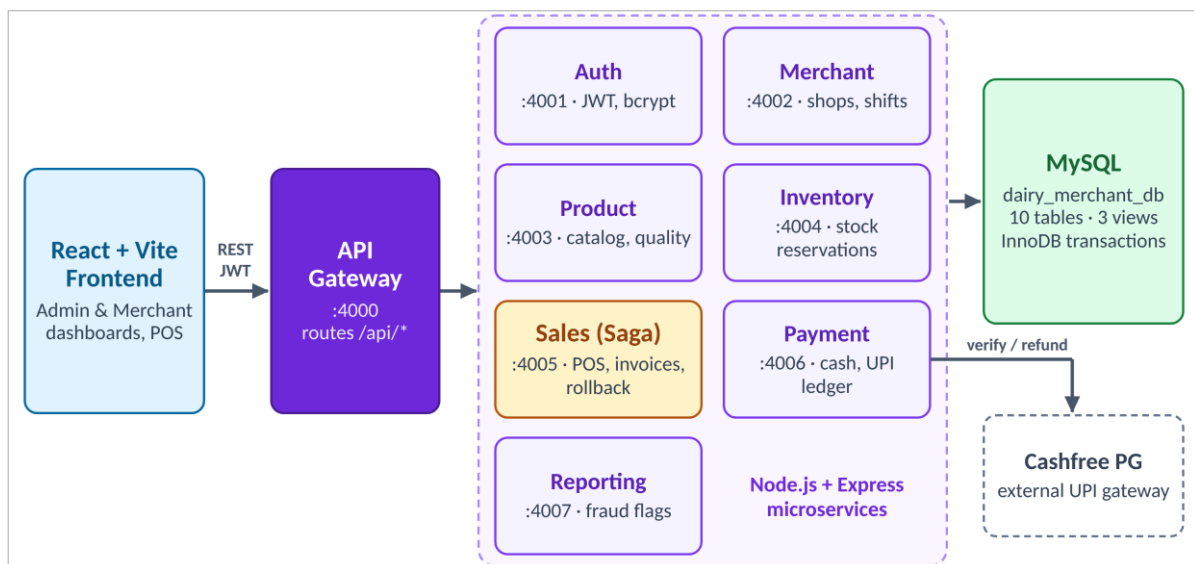


Figure 5.1: Overall system architecture

Client tier. The React application runs in the browser. It renders role-specific dashboards and sends JSON requests to /api/* endpoints with the user's JWT.

Gateway tier. The API gateway (port 4000) is the single entry point. It inspects the URL prefix and forwards the request, including headers and body, to the responsible service, then relays the response. If a downstream service is unreachable it returns HTTP 502 Bad Gateway instead of hanging.

Service tier. Seven services each own one business capability:

Table 5.1: Microservices and their responsibilities

Service	Port	Routes	Responsibility
Auth	4001	/api/auth	Login, token issuance, current-user profile
Merchant	4002	/api/merchants	Merchant directory, shifts, activation status
Product	4003	/api/products	Product catalogue, official prices, quality tests
Inventory	4004	/api/inventory	Stock inflow, current stock, reservations with TTL
Sales	4005	/api/sales, /api/invoices	POS orders, saga orchestration, invoices, recovery

Service	Port	Routes	Responsibility
Payment	4006	/api/payments	Payment ledger, Cashfree create/verify/refund
Reporting	4007	/api/discrepancies	Dashboards, revenue and discrepancy reports

Data tier. All services connect to the dairy_merchant_db database. Each service reads and writes only the tables of its own domain, apart from the reporting service, which reads the views.

5.2 System Workflow

The most important workflow is recording a sale at the point of sale. It involves four services and the external payment gateway. Figure 5.2 shows the sequence of calls for a successful sale.

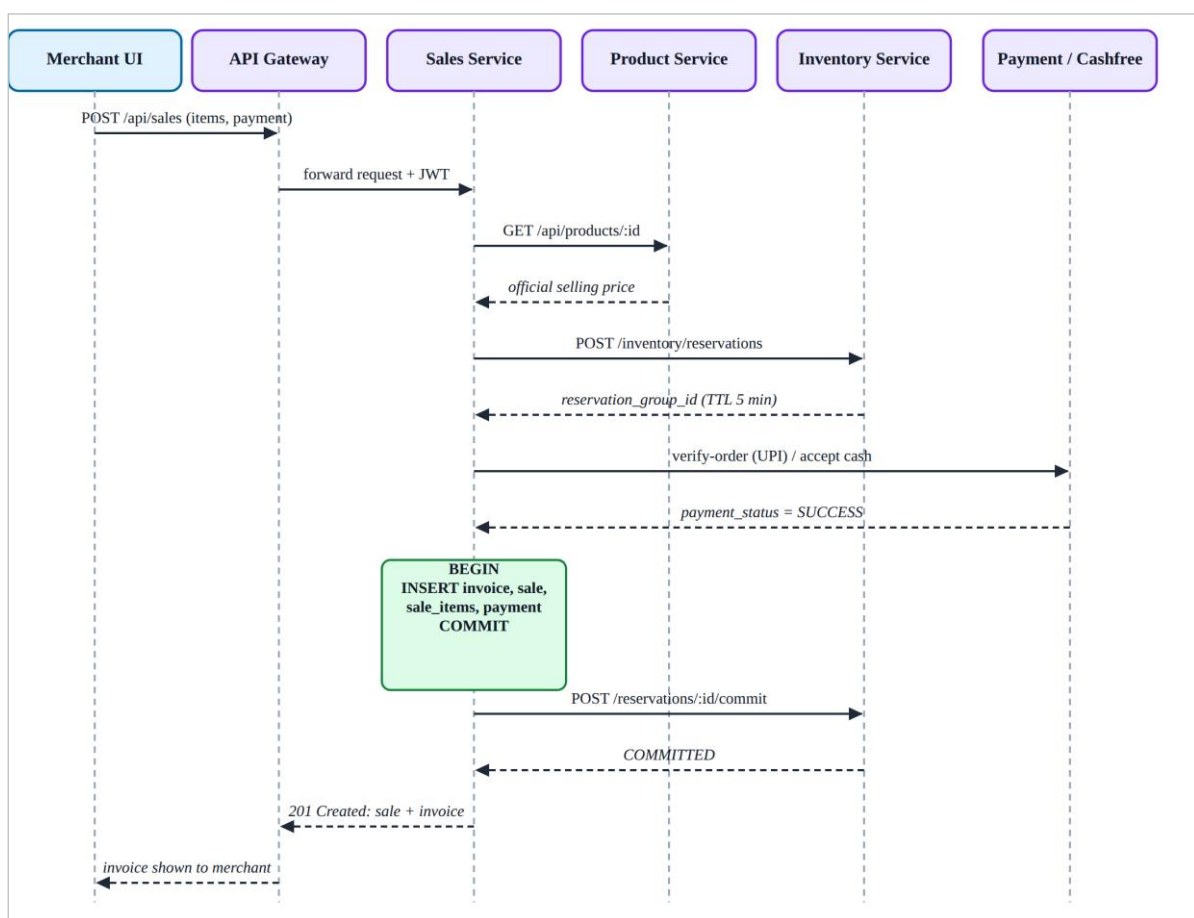


Figure 5.2: Sequence of service calls for a successful sale

Other workflows are simpler and involve a single service:

- **Login:** the user submits credentials; the auth service verifies them and returns a JWT and profile; the frontend redirects to the role's dashboard.
- **Record stock in:** a merchant selects a product, enters the received quantity and cost per unit; the inventory service inserts a stock_entries row and increases

inventory.quantity_available in one transaction.

- **Manage catalogue:** an admin adds or edits a product, or records a quality test with fat %, SNF % and grade.
- **Review discrepancies:** an admin opens the Discrepancies page; the reporting service queries the stock and payment discrepancy views and returns only the rows with a non-zero difference.

5.3 Flowchart / Data Flow

The flowchart in Figure 5.3 shows the decision logic of the sale saga, including the compensating paths taken when a step fails.

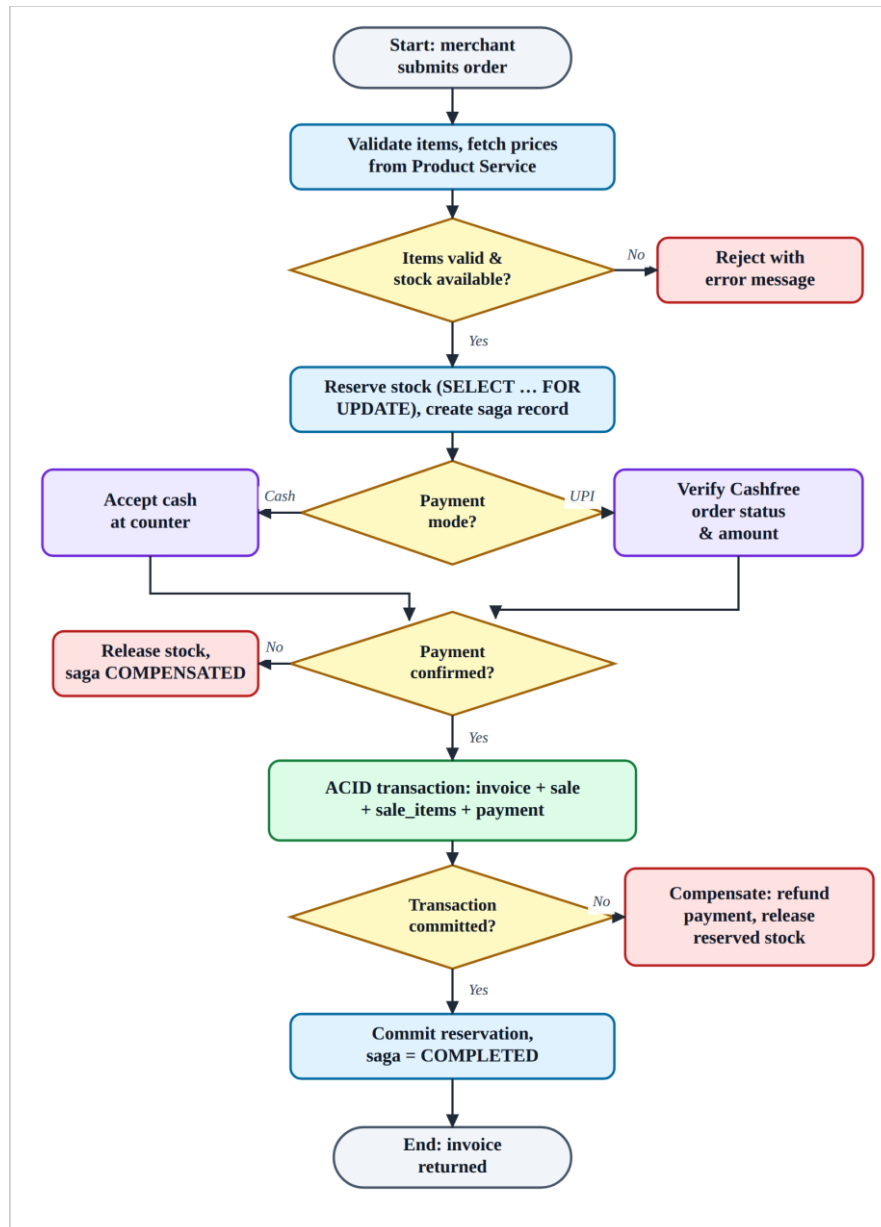


Figure 5.3: Flowchart of the point-of-sale process

The context-level data flow diagram (Figure 5.4) shows the system as a single process exchanging data with its three external entities.

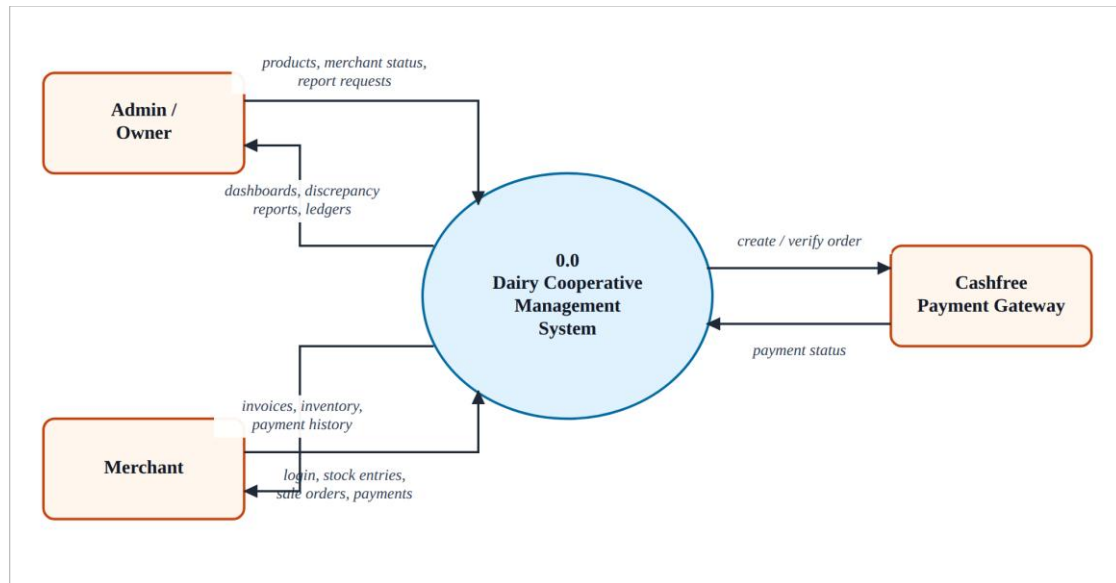


Figure 5.4: Level-0 data flow diagram (context diagram)

The Level-1 DFD (Figure 5.5) decomposes the system into six processes and five data stores.

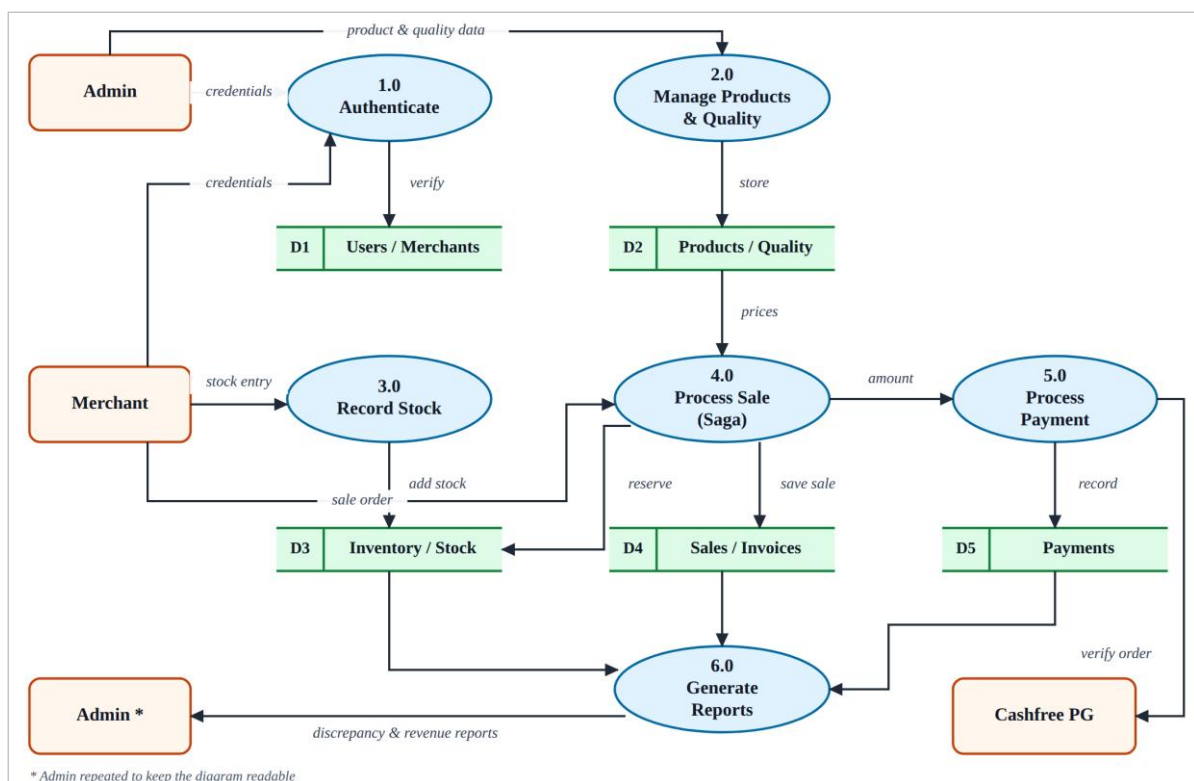


Figure 5.5: Level-1 data flow diagram

5.4 Module Description

Table 5.2: Functional modules

Module	Users	Description
Authentication	All	Login with email, merchant code or merchant id; JWT issuance; role-based redirection

Module	Users	Description
Merchant Management	Admin	List merchants with shop, shift and contact details; activate or deactivate accounts
Product Catalogue & Quality	Admin; Merchant (read)	Maintain products, units and official selling prices; record fat %, SNF % and grade per test
Stock & Inventory	Merchant; Admin (read)	Record incoming stock with cost; maintain available quantity per merchant and product; low-stock view
Point of Sale	Merchant	Multi-item billing with tax and discount; cash or Cashfree UPI; invoice generation through the saga
Payments	Merchant; Admin	Payment ledger with method, status and transaction reference; Cashfree order, verification and refund
Invoices	Merchant; Admin	One invoice per sale with a unique invoice number
Reporting & Discrepancies	Admin, Owner; Merchant (summary)	Executive dashboard, merchant revenue, stock and payment discrepancy reports
Recovery	Admin, Owner	Recovery sweep for interrupted sagas and detection of orphan payments

5.5 Input and Output Design

Inputs are captured through forms that validate data on the client and again on the server.

Outputs are presented as dashboards, tables and status badges.

Table 5.3: Input design

Screen	Inputs	Validation
Login	Merchant ID / email, access code / password	Both required; account must be ACTIVE
Record Stock In	Product, quantity, cost per unit, date	Quantity > 0, cost ≥ 0, product must exist
Point of Sale	Line items (product, quantity), tax, discount, payment mode, cash tendered	At least one item; quantity > 0; total > 0; price taken from server; stock must be available
Add Product	Name, category, unit, selling price	Name required; price ≥ 0
Quality Test	Product, fat %, SNF %, grade, test date	Grade must be A, B or C

Table 5.4: Output design

Output	Shown to	Content
Executive Overview	Admin, Owner	Registered merchants, gross sales billed, collected revenue, payment and stock flag counts
Shop Performance Overview	Merchant	Today's transactions and revenue, pending payments, current stock, recent sales
Invoice / sale confirmation	Merchant	Invoice number, line items, totals and payment status

Output	Shown to	Content
Payment Ledger	Admin, Merchant	Payment id, sale, method, amount, transaction reference, status
Discrepancy reports	Admin, Owner	Payment variance per merchant; stock variance per merchant and product

5.6 Database Design

Figure 5.6 shows the entity-relationship design. The schema contains ten core tables; their keys and relationships are shown below.

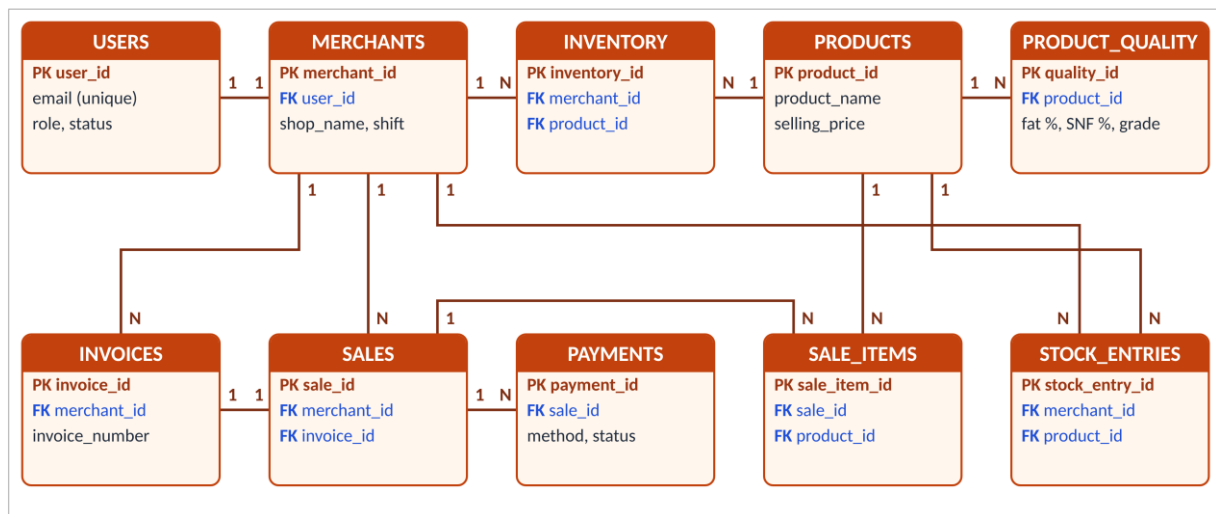


Figure 5.6: Entity-relationship diagram of dairy_merchant_db

Table 5.5: Core tables of the database

Table	Primary key	Important columns and constraints
users	user_id	email UNIQUE, password_hash, role ENUM(ADMIN, MERCHANT, OWNER), status
merchants	merchant_id	user_id UNIQUE FK, merchant_code UNIQUE, shift ENUM, shop_name, phone, address
products	product_id	product_name, category, unit, selling_price CHECK ≥ 0 , status
product_quality	quality_id	product_id FK, fat_percent, snf_percent, grade ENUM(A, B, C), tested_date
stock_entries	stock_entry_id	merchant_id FK, product_id FK, quantity CHECK > 0 , cost_per_unit, total_cost (generated)
inventory	inventory_id	merchant_id FK, product_id FK, quantity_available CHECK ≥ 0 , UNIQUE(merchant_id, product_id)
invoices	invoice_id	merchant_id FK, invoice_number UNIQUE, invoice_date, total_amount
sales	sale_id	merchant_id FK, invoice_id UNIQUE FK, subtotal, tax, discount, total_amount, sale_status

Table	Primary key	Important columns and constraints
sale_items	sale_item_id	sale_id FK, product_id FK, quantity CHECK > 0, unit_price CHECK ≥ 0
payments	payment_id	sale_id FK, payment_method ENUM, amount, transaction_ref UNIQUE, payment_status ENUM

The main relationships are: a user has at most one merchant profile (1:1); a merchant has many stock entries, inventory rows, invoices and sales (1:N); a sale has one invoice (1:1), many sale items (1:N) and its payments (1:N); and a product appears in many quality tests, stock entries, inventory rows and sale items (1:N). Inventory and stock_entries are associative entities that resolve the many-to-many relationship between merchants and products.

CHAPTER -6 IMPLEMENTATION

6.1 Module-wise Implementation

Authentication module (auth-service)

The login endpoint accepts an identifier that may be an email address, a merchant code (for example M1) or a numeric merchant id. A single query joins users with merchants so that any of these identifiers resolves to the same account. The password is checked with bcrypt, inactive accounts are rejected with HTTP 403, and on success a JWT and the user profile (including shop name and shift for merchants) are returned.

Merchant module (merchant-service)

Admins can list all merchants with their shop, shift, contact details and status, view a single merchant, and change a merchant's status. Deactivated merchants can no longer log in because the auth service checks users.status on every login.

Product and quality module (product-service)

This service manages the product catalogue: name, category, unit of measure and the official selling price. Merchants can read the catalogue, while only admins can create, update or discontinue products. The quality endpoints store laboratory results per product (fat percentage, solids-not-fat percentage, grade and test date), which supports the cooperative's quality-based grading.

Inventory module (inventory-service)

Stock inflow is recorded in stock_entries, and the running balance in inventory. Recording a stock entry and increasing the balance happen in one transaction, using INSERT ... ON DUPLICATE KEY UPDATE on the unique (merchant_id, product_id) pair. The service also exposes the reservation API used by the sale saga: create a reservation group, commit it once the sale is saved, or release it on failure. A background job runs every ten seconds and returns stock from reservations whose five-minute TTL has expired.

Sales and invoice module (sales-service)

The sales service orchestrates the POS saga described in Section 6.2. It validates line items, fetches official prices, computes subtotal, tax, discount and total on the server, coordinates the reservation and payment, and finally writes the invoice, sale, line items and payment in one transaction. It supports an idempotency key so that a retried request returns the original result instead of creating a duplicate sale. A recovery job runs every thirty seconds to resume or

compensate sagas that were interrupted, and an orphan-payment endpoint lists payments that have no completed sale.

Payment module (payment-service)

The payment service keeps the payment ledger and wraps the Cashfree Orders API. It creates an order for a given amount, verifies an order's status after checkout, and issues refunds when the saga needs to compensate a successful payment.

Reporting module (reporting-service)

The reporting service builds the admin executive summary, the merchant summary, merchant revenue, and the stock and payment discrepancy reports by querying the three SQL views described in Section 6.7.

6.2 Algorithm / Pseudocode

The core algorithm of the system is the orchestrated sale saga executed by the sales service. Figure 6.1 shows the states a saga passes through and the transitions taken on failure.

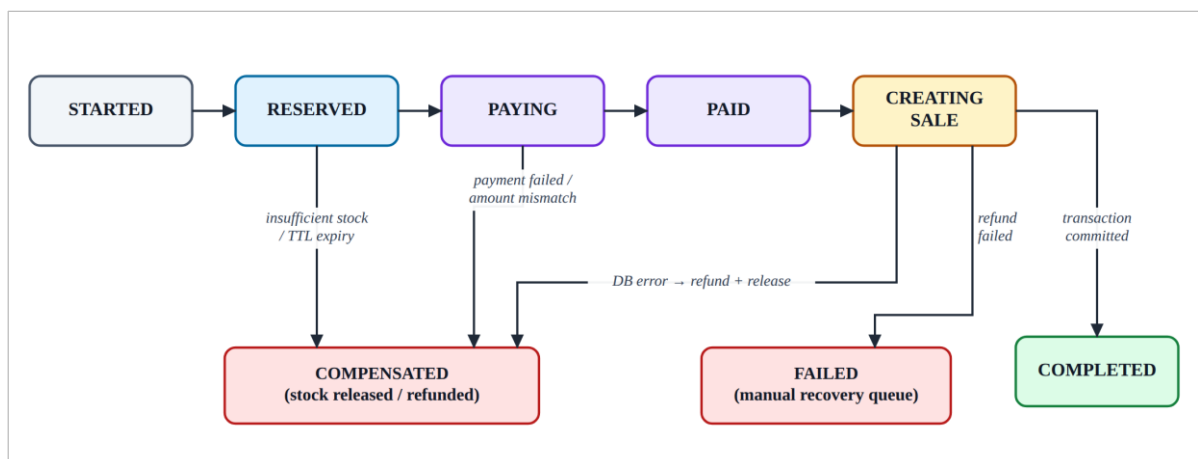


Figure 6.1: Saga state transitions for a sale

Listing 6.1: Pseudocode of the sale saga

```
ALGORITHM RecordSale(merchant, items, tax, discount, paymentMethod, idempotencyKey)
1  IF a saga exists for idempotencyKey THEN
2    RETURN its stored result                                // safe retry
3  FOR EACH item IN items
4    price <- ProductService.getPrice(item.product_id)    // never trust client price
5    subtotal <- subtotal + price * item.quantity
6  total <- subtotal + tax - discount
7  IF total <= 0 THEN REJECT "invalid total"
8  group <- InventoryService.reserve(merchant, items, ttl = 300 s)
9    // inside: SELECT quantity_available ... FOR UPDATE; decrement; log reservation
10 IF reservation failed THEN REJECT "insufficient stock"
11 saga.state <- PAYING
12 IF paymentMethod = UPI THEN
13   status <- PaymentService.verifyOrder(orderId)
14   IF status != SUCCESS OR amount mismatch THEN
15     InventoryService.release(group); saga.state <- COMPENSATED
16     REJECT "payment not confirmed"
17 saga.state <- PAID
18 BEGIN TRANSACTION
19   INSERT invoice; INSERT sale; INSERT sale_items; INSERT payment
20 IF COMMIT fails THEN
21   ROLLBACK
22   refund payment (UPI); InventoryService.release(group)
23   saga.state <- COMPENSATED (or FAILED if the refund also fails)
24   REJECT "sale could not be recorded"
25 InventoryService.commit(group, saleId)
26 saga.state <- COMPLETED
27 RETURN sale, invoice
```

The time complexity of the algorithm is $O(n)$ in the number of line items, since each item requires one price lookup, one locked row read and a constant number of inserts. All database work for a step is short and bounded, which keeps row locks held only briefly.

6.3 Core Back-End Implementation (Node.js / Express)

Listing 6.2 shows the API gateway's forwarding function. Every route prefix is mapped to a service URL, and the request is forwarded with its method, headers and body using the built-in fetch API.

Listing 6.2: API gateway request forwarding (simplified from services/api-gateway/index.js)

```
async function proxyRequest(targetBaseUrl, req, res) {
  const targetUrl = `${targetBaseUrl}${req.originalUrl}`;
  const headers = { ...req.headers };
  delete headers['host'];
  delete headers['content-length'];
  const options = { method: req.method, headers };
  if (req.method !== 'GET' && req.method !== 'HEAD' && req.body
    && Object.keys(req.body).length > 0) {
    options.body = JSON.stringify(req.body);
    options.headers['content-type'] = 'application/json';
  }
  try {
    const downstreamRes = await fetch(targetUrl, options);
    res.status(downstreamRes.status);
    const data = await downstreamRes.json();
    return res.json(data);
  } catch (err) {
    if (err.cause?.code === 'ECONNREFUSED')
      return res.status(502).json({ message: 'Bad Gateway: Downstream service is
currently unreachable' });
    return res.status(500).json({ message: 'Gateway proxy error' });
  }
}
app.use('/api/auth', (req, res) => proxyRequest(AUTH_SERVICE_URL, req, res));
app.use('/api/sales', (req, res) => proxyRequest(SALES_SERVICE_URL, req, res));
```

Listing 6.3 shows the authentication middleware shared by the services. It extracts the Bearer token, verifies its signature and expiry, and attaches the decoded user to the request; `authorizeRole` then checks the role for protected endpoints.

Listing 6.3: JWT verification middleware

```
function authenticateToken(req, res, next) {
  const header = req.headers.authorization;
  if (!header || !header.startsWith('Bearer ')) {
    return res.status(401).json({ message: 'No token provided' });
  }
  const token = header.split(' ')[1];
  try {
    req.user = jwt.verify(token, JWT_SECRET); // { user_id, role, name }
    next();
  } catch (err) {
    return res.status(401).json({ message: 'Invalid or expired token' });
  }
}
```

Listing 6.4 shows how the auth service resolves a login identifier with a single parameterised query.

Listing 6.4: Login lookup in the auth service

```
const [rows] = await pool.query(
  `SELECT u.user_id, u.name, u.email, u.password_hash, u.role, u.status,
    m.merchant_id, m.merchant_code, m.shop_name, m.shift
    FROM users u
    LEFT JOIN merchants m ON m.user_id = u.user_id
    WHERE u.email = ?
      OR m.merchant_code = ?
      OR (m.merchant_id = ? AND ? REGEXP '^[0-9]+$')`,
  [cleanId, cleanId, isNaN(cleanId) ? 0 : Number(cleanId), cleanId]
);
if (rows.length === 0) return res.status(401).json({ message: 'Invalid credentials' });
const user = rows[0];
if (user.status !== 'ACTIVE') return res.status(403).json({ message: 'Account
deactivated' });
const match = await bcrypt.compare(cleanPass, user.password_hash);
```

6.4 REST APIs Used

Table 6.1 lists the main endpoints exposed through the gateway. All endpoints except login require a valid JWT.

Table 6.1: Main REST endpoints

Method	Endpoint	Roles	Purpose
POST	/api/auth/login	Public	Authenticate and receive a JWT
GET	/api/auth/me	Any	Current user profile
GET	/api/merchants	Admin	List merchants
PUT	/api/merchants/:id/status	Admin	Activate or deactivate a merchant
GET / POST	/api/products	Any / Admin	Read catalogue / add product
PUT / DELETE	/api/products/:id	Admin	Update or discontinue product
GET / POST	/api/products/:id/quality	Any / Admin	Read / record quality test
GET	/api/inventory	Admin, Merchant	Current stock (own stock for merchants)
GET	/api/inventory/low-stock	Admin, Merchant	Products below a threshold
POST	/api/inventory/stock	Merchant	Record incoming stock
POST	/api/inventory/reservations	Internal (saga)	Reserve stock with TTL
POST	/api/inventory/reservations/:id/commit	Internal (saga)	Commit a reservation to a sale
POST	/api/inventory/reservations/:id/release	Internal (saga)	Release reserved stock
GET / POST	/api/sales	Any / Merchant, Admin	List sales / record a sale (saga)
GET	/api/sales/:id	Any	Sale with its line items
GET	/api/invoices	Any	List invoices
GET	/api/sales/orphan-payments	Admin, Owner	Payments without a completed sale

Method	Endpoint	Roles	Purpose
POST	/api/sales/recovery-sweep	Admin, Owner	Resume or compensate interrupted sagas
GET	/api/payments	Admin, Merchant	Payment ledger
POST	/api/payments/cashfree/create-order	Merchant	Create Cashfree order
POST	/api/payments/cashfree/verify-order	Merchant / saga	Verify order status
POST	/api/payments/cashfree/refund	Saga	Refund a payment (compensation)
GET	/api/discrepancies/summary/admin	Admin, Owner	Executive dashboard summary
GET	/api/discrepancies/summary/merchant	Merchant	Merchant dashboard summary
GET	/api/discrepancies/stock	Admin, Owner	Stock discrepancy report
GET	/api/discrepancies/payment	Admin, Owner	Payment discrepancy report

6.5 Microservices Implementation

Each service is an independent Node.js process with its own dependencies, port and environment file, and exposes a /health endpoint used to check that it is running. Services call one another over HTTP using the internal service URLs configured in their .env files (for example `PRODUCT_SERVICE_URL=http://localhost:4003`), and forward the caller's JWT so that the downstream service applies the same authorisation rules.

The services were extracted from the original monolith in steps. During migration, the gateway forwarded routes that had not yet been extracted to the monolith; after every route had a dedicated service, the frontend was switched to the gateway, a stabilisation period confirmed that no traffic reached the monolith, and the monolith route was decommissioned. The repository's integration suites (steps 9 to 17) document and test each stage of this migration.

6.6 Transactions and Concurrency Control (Saga Implementation)

Listing 6.5 shows the reservation step in the inventory service. Each requested product's inventory row is read with FOR UPDATE inside a transaction, so concurrent reservations for the same product are serialised. The quantity is deducted immediately and a reservation record with an expiry time is written, together with a saga record for recovery.

Listing 6.5: Stock reservation with row-level locking (inventory-service)

```
await conn.beginTransaction();
for (const item of validatedItems) {
  const [invRows] = await conn.query(
    'SELECT quantity_available FROM inventory WHERE merchant_id = ? AND product_id = ?
    FOR UPDATE',
    [merchant_id, item.product_id]);
  const available = invRows.length ? Number(invRows[0].quantity_available) : 0;
  if (available < item.quantity) {
    throw { status: 400, message: `Insufficient inventory for product_id
    ${item.product_id}` };
  }
  await conn.query( // deduct immediately to prevent overselling
    'UPDATE inventory SET quantity_available = quantity_available - ? WHERE merchant_id =
    ? AND product_id = ?',
    [item.quantity, merchant_id, item.product_id]);
  await conn.query(
    `INSERT INTO inventory_reservations
    (reservation_group_id, merchant_id, product_id, quantity_reserved, status,
    expires_at)
    VALUES (?, ?, ?, ?, 'RESERVED', ?)` ,
    [reservation_group_id, merchant_id, item.product_id, item.quantity, expires_at]);
}
await conn.query(
  `INSERT INTO saga_instances (saga_id, reservation_group_id, merchant_id, current_step,
  saga_status, payload)
  VALUES (?, ?, ?, 'RESERVED', 'RESERVED', ?)` ,
  [`saga_${reservation_group_id}`, reservation_group_id, merchant_id, JSON.stringify({
  items: validatedItems, expires_at })]);
await conn.commit();
```

Listing 6.6 shows the local ACID transaction in the sales service that records the sale once stock is reserved and payment is confirmed.

Listing 6.6: Atomic recording of a sale (sales-service)

```
const conn = await pool.getConnection();
try {
  await conn.beginTransaction();
  const [invResult] = await conn.query(
    'INSERT INTO invoices (merchant_id, invoice_number, invoice_date, total_amount)
    VALUES (?, ?, CURDATE(), ?)',
    [merchantId, invoiceNumber, total_amount]);
  const [saleResult] = await conn.query(
    'INSERT INTO sales (merchant_id, invoice_id, subtotal, tax, discount, total_amount,
    sale_status) VALUES (?, ?, ?, ?, ?, ?, "COMPLETED")',
    [merchantId, invResult.insertId, subtotal, numTax, numDiscount, total_amount]);
  const saleId = saleResult.insertId;
  for (const item of validatedItems) {
    await conn.query(
      'INSERT INTO sale_items (sale_id, product_id, quantity, unit_price) VALUES (?, ?,
      ?, ?)',
      [saleId, item.product_id, item.quantity, item.unit_price]);
  }
  await conn.query(
    'INSERT INTO payments (sale_id, payment_method, amount, transaction_ref,
    payment status) VALUES (?, ?, ?, ?, ?)',
    [saleId, payment_method, total_amount, finalTxnRef, finalPaymentStatus]);
  await conn.commit();
} catch (err) {
  await conn.rollback(); // then compensate: refund payment, release reservation
} finally {
  conn.release();
}
```

Recovery is handled by two background jobs. The inventory service expires reservations whose TTL has passed and returns their quantity to inventory every ten seconds. The sales service scans every thirty seconds for sagas left in the STARTED, RESERVED or PAID state and either completes or compensates them, so that a crash in the middle of a sale does not leave stock locked or a payment unaccounted for.

6.7 Database Implementation (Views, Indexes and Constraints)

Listing 6.7 shows one of the core tables. CHECK constraints guard against negative amounts, the ENUM restricts the status values, and the foreign keys tie each sale to its merchant and invoice.

Listing 6.7: Definition of the sales table (database/schema.sql)

```
CREATE TABLE sales (
  sale_id      INT AUTO_INCREMENT PRIMARY KEY,
  merchant_id  INT NOT NULL,
  invoice_id   INT UNIQUE,
  sale_date    DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  subtotal     DECIMAL(12,2) NOT NULL CHECK (subtotal >= 0),
  tax          DECIMAL(12,2) NOT NULL DEFAULT 0,
  discount     DECIMAL(12,2) NOT NULL DEFAULT 0,
  total_amount DECIMAL(12,2) NOT NULL CHECK (total_amount >= 0),
  sale_status  ENUM('COMPLETED','CANCELLED') NOT NULL DEFAULT 'COMPLETED',
  CONSTRAINT fk_sale_merchant FOREIGN KEY (merchant_id) REFERENCES
merchants(merchant_id) ON DELETE CASCADE,
  CONSTRAINT fk_sale_invoice  FOREIGN KEY (invoice_id)  REFERENCES invoices(invoice_id)
ON DELETE SET NULL
) ENGINE=InnoDB;
```

The stock discrepancy view (Listing 6.8) compares, for every merchant and product, the stock that should exist (total incoming minus total sold) with the stock actually recorded in inventory. Incoming and sold quantities are aggregated separately in derived tables before the join, so neither total is multiplied by the other.

Listing 6.8: Stock discrepancy view

```
CREATE OR REPLACE VIEW vw_stock_discrepancy AS
SELECT i.merchant_id, i.product_id, pr.product_name,
       COALESCE(se.total_incoming, 0) AS total_incoming,
       COALESCE(si.total_sold, 0) AS total_sold,
       (COALESCE(se.total_incoming, 0) - COALESCE(si.total_sold, 0)) AS expected_stock,
       i.quantity_available AS actual_stock,
       ((COALESCE(se.total_incoming, 0) - COALESCE(si.total_sold, 0))
        - i.quantity_available) AS discrepancy
FROM inventory i
JOIN products pr ON pr.product_id = i.product_id
LEFT JOIN (SELECT merchant_id, product_id, SUM(quantity) AS total_incoming
           FROM stock_entries GROUP BY merchant_id, product_id) se
  ON se.merchant_id = i.merchant_id AND se.product_id = i.product_id
LEFT JOIN (SELECT s.merchant_id, it.product_id, SUM(it.quantity) AS total_sold
           FROM sale_items it JOIN sales s ON s.sale_id = it.sale_id
           WHERE s.sale_status = 'COMPLETED'
           GROUP BY s.merchant_id, it.product_id) si
  ON si.merchant_id = i.merchant_id AND si.product_id = i.product_id;
```

The payment discrepancy view applies the same idea to money: for each merchant it subtracts the total of SUCCESS payments from the total billed in completed sales. A third view, `vw_merchant_revenue`, totals sales and collected revenue per merchant for the dashboards.

The saga relies on two supporting tables used by the inventory and sales services. Their structure, as used by the service code, is shown in Listing 6.9.

Listing 6.9: Supporting tables for the saga

```
CREATE TABLE inventory_reservations (
  reservation_id      INT AUTO_INCREMENT PRIMARY KEY,
  reservation_group_id VARCHAR(64) NOT NULL,
  merchant_id         INT NOT NULL,
  product_id          INT NOT NULL,
  quantity_reserved   DECIMAL(10,2) NOT NULL CHECK (quantity_reserved > 0),
  status              ENUM('RESERVED', 'COMMITTED', 'RELEASED', 'EXPIRED') NOT NULL DEFAULT
'RESERVED',
  sale_id             INT NULL,
  expires_at          DATETIME NOT NULL,
  created_at          TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  INDEX idx_res_group (reservation_group_id),
  INDEX idx_res_expiry (status, expires_at)
) ENGINE=InnoDB;

CREATE TABLE saga_instances (
  saga_id             VARCHAR(80) PRIMARY KEY,
  reservation_group_id VARCHAR(64) UNIQUE,
  merchant_id         INT NOT NULL,
  current_step        VARCHAR(50) NOT NULL,
  saga_status         ENUM('STARTED', 'RESERVED', 'PAID', 'COMPLETED', 'COMPENSATED', 'FAILED') NOT
NULL,
  payload             JSON,
  error_log           TEXT,
  created_at          TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at          TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
) ENGINE=InnoDB;
```

6.8 User Interface Implementation

The user interface is implemented as React function components using hooks (`useState`, `useEffect`). A shared `Layout` component renders the dark sidebar with role-specific navigation, the page header with the user's role and shift badges, and a sign-out control. Each page fetches its data on mount through the `Axios` instance and renders tables, stat cards or forms.

The point-of-sale page is the most complex component. It maintains a list of order lines, recalculates the grand total whenever a quantity, tax or discount changes, and switches between two payment panels: a Cashfree UPI panel, which creates an order and opens Cashfree's checkout, and a cash panel, which computes the change to return from the cash tendered. On submission it sends the order to `/api/sales` and shows the resulting invoice or the error returned by the saga.

Status values are shown as coloured badges (for example green for `SUCCESS` and red for `FAILED`), and discrepancy rows are highlighted so that administrators can spot problems at a glance. The full set of screens is presented in Chapter 7.

CHAPTER -7 EXECUTION AND DEMONSTRATION

7.1 Installation and Database Setup

The project requires Node.js and MySQL 8.0 to be installed. The database is created from the schema file, which also inserts seed data (one admin, two owners, two merchants, five products, stock entries, three sales and their payments).

Listing 7.1: Creating the database

```
# from the project root
mysql -u root -p < database/schema.sql

# verify
mysql -u root -p -e "USE dairy_merchant_db; SHOW TABLES;"
```

Each service reads its configuration from a .env file in its folder. The required variables are the port, the database connection, the JWT secret shared by all services, and, for the payment service, the Cashfree sandbox credentials.

Listing 7.2: Service configuration (.env) – values are examples

```
PORT=4005
DB_HOST=localhost
DB_USER=root
DB_PASSWORD=<your MySQL password>
DB_NAME=dairy_merchant_db
JWT_SECRET=<long random secret, identical in every service>
PRODUCT_SERVICE_URL=http://localhost:4003
INVENTORY_SERVICE_URL=http://localhost:4004
PAYMENT_SERVICE_URL=http://localhost:4006
# payment-service only
CASHFREE_APP_ID=<sandbox app id>
CASHFREE_SECRET_KEY=<sandbox secret>
CASHFREE_ENV=TEST
```

7.2 Execution Commands

Listing 7.3: Installing dependencies and starting the system

```
# install dependencies for every service and the frontend
for d in services/*/ frontend; do (cd "$d" && npm install); done

# start the gateway, all seven services and the Vite dev server
npm start # runs node start_all.js

# individual service (for debugging)
cd services/sales-service && node index.js

# health checks
curl http://localhost:4000/health
curl http://localhost:4005/api/sales/health

# run the final integration suite (services and MySQL must be running)
npm test
```

After start-up the application is available at <http://localhost:5173>. The gateway listens on port 4000 and the services on ports 4001 to 4007.

7.3 Sample Inputs and Outputs

Table 7.1 lists representative inputs and the output the system produces for each, based on the implemented validation and business logic and the seed data.

Table 7.1: Sample inputs and expected outputs

#	Input	Expected output
1	Login with merchant code M1 and correct access code	HTTP 200; JWT and merchant profile; redirect to merchant Overview
2	Login with a wrong password	HTTP 401 "Invalid Merchant ID/Email or Access Code"
3	Login to a deactivated merchant account	HTTP 403 "This account has been deactivated"
4	Record stock: Fresh Curd, quantity 20, cost ₹50	stock_entries row inserted; Fresh Curd inventory increases by 20
5	Record stock with quantity 0	HTTP 400 "Quantity must be a positive number"; nothing saved
6	Cash sale: 10 L milk + 2 kg paneer, no tax or discount	Total ₹1,190.00 computed on server; invoice created; inventory reduced by 10 and 2
7	Sale with milk price edited to ₹1 in the browser	Server ignores client price and bills at the catalogue price of ₹55
8	Sale of 500 L milk when 60 L are available	HTTP 400 "Insufficient inventory for product_id 1 (Available: 60, Requested: 500)"; nothing saved
9	Same sale submitted twice with one idempotency key	Second request returns the first sale; no duplicate invoice
10	UPI sale whose Cashfree order is not paid	Reservation released; saga COMPENSATED; HTTP 400 "Cashfree payment verification failed"
11	Admin opens Discrepancies (seed data)	Koteshwar Dairy Parlour: payment variance ₹700.00; Full Cream Milk stock variance 10 units
12	Merchant calls /api/discrepancies/stock	HTTP 403 (endpoint restricted to Admin and Owner)

7.4 Screenshots of Execution

The following screenshots show the running application with the seed data loaded.

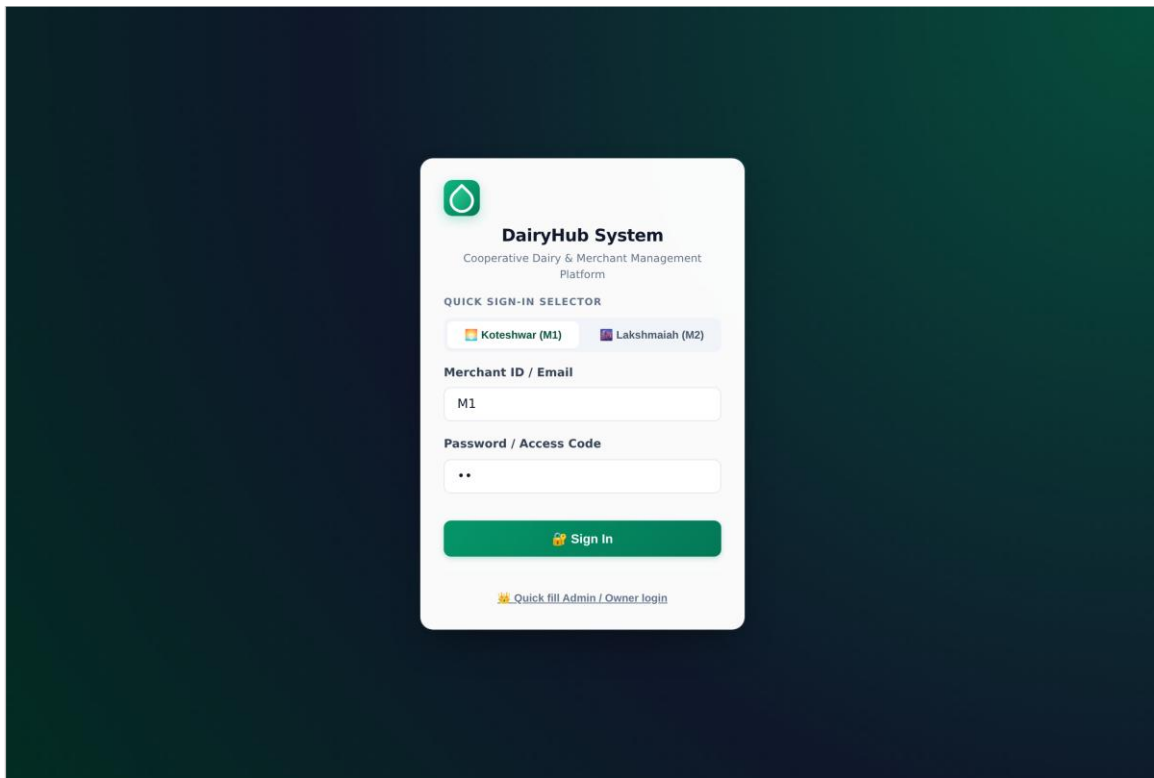


Figure 7.1: Login page with merchant quick-login options

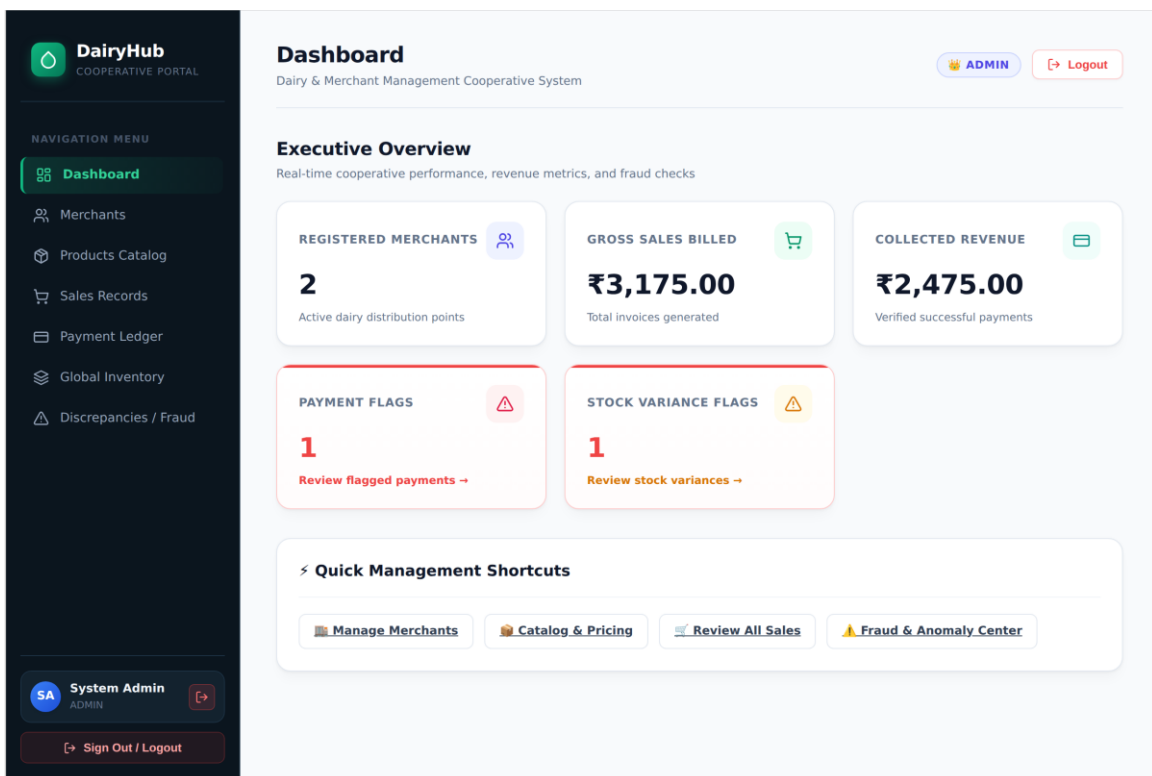


Figure 7.2: Admin executive overview with revenue and fraud flags


DairyHub
COOPERATIVE PORTAL

NAVIGATION MENU
 Dashboard
Merchants
 Products Catalog
 Sales Records
 Payment Ledger
 Global Inventory
 Discrepancies / Fraud

 SA System Admin ADMIN
 Sign Out / Logout

Merchants

Dairy & Merchant Management Cooperative System

ADMIN Logout

Merchants Directory

Manage registered dairy franchise outlets, contact records, and access status

Q Search merchants...

MERCHANT OPERATOR	MERCHANT ID	ASSIGNED SHIFT	FRANCHISE COUNTER / SHOP	PHONE CONTACT	EMAIL ACCOUNT
 Koteswar Rao	M1		Koteswar Dairy Parlour	9876543210	koteswar.merchant@dairycoop.cc
 Lakshmaiah Setty	M2		Lakshmaiah Milk Center	9876543211	lakshmaiah.merchant@dairycoop.c

Figure 7.3: Merchants directory


DairyHub
COOPERATIVE PORTAL

NAVIGATION MENU
 Dashboard
 Merchants
Products Catalog
 Sales Records
 Payment Ledger
 Global Inventory
 Discrepancies / Fraud

 SA System Admin ADMIN
 Sign Out / Logout

Products Catalog

Dairy & Merchant Management Cooperative System

ADMIN Logout

Product Catalog & Base Pricing

Maintain standardized milk products, standardized units, and standard cooperative retail rates

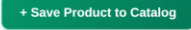

Add New Catalog Product

Product Name

Category

Measurement Unit

Selling Price (₹)



Active Products List

Q Search products...

PRODUCT NAME	CATEGORY	MEASUREMENT UNIT	OFFICIAL RETAIL PRICE	STATUS
 Full Cream Milk		Litre	₹55.00	
 Fresh Curd		Kg	₹70.00	
 Farm Paneer		Kg	₹320.00	

Figure 7.4: Product catalogue management


DairyHub
COOPERATIVE PORTAL

NAVIGATION MENU

- Dashboard
- Merchants
- Products Catalog
- Sales Records**
- Payment Ledger
- Global Inventory
- Discrepancies / Fraud

SA System Admin
ADMIN

Sign Out / Logout

Sales Records

Dairy & Merchant Management Cooperative System

Master Sales Registry

Complete registry of transactions across all merchant franchise shops

Search by ID, Shop or Status...

SALE ID	SHOP OUTLET	DATE & TIME	SUBTOTAL	TAX	DISCOUNT	TOTAL BILL	STATUS
#1	Koteshwar Dairy Parlour	29/8/2026, 9:15:00 am	₹1600.00	₹50.00	₹0.00	₹1650.00	COMPLETED
#2	Koteshwar Dairy Parlour	29/8/2026, 11:30:00 am	₹700.00	₹0.00	₹0.00	₹700.00	COMPLETED
#3	Lakshmaiah Milk Center	29/8/2026, 10:00:00 am	₹800.00	₹25.00	₹0.00	₹825.00	COMPLETED

Figure 7.5: Master sales registry across all outlets


DairyHub
COOPERATIVE PORTAL

NAVIGATION MENU

- Dashboard
- Merchants
- Products Catalog
- Sales Records
- Payment Ledger**
- Global Inventory
- Discrepancies / Fraud

SA System Admin
ADMIN

Sign Out / Logout

Payment Ledger

Dairy & Merchant Management Cooperative System

Master Payment Ledger

Comprehensive collection logs, digital transaction IDs, and settlement status

Search payments, shops, refs...

PAYMENT ID	FRANCHISE SHOP	SALE ID	PAYMENT MODE	AMOUNT	TXN REFERENCE	STATUS
#1	Koteshwar Dairy Parlour	Sale #1	UPI	₹1650.00	TXN-90001	SUCCESS
#2	Koteshwar Dairy Parlour	Sale #2	CASH	₹700.00	—	FAILED
#3	Lakshmaiah Milk Center	Sale #3	CARD	₹825.00	TXN-90003	SUCCESS

Figure 7.6: Payment ledger showing a failed cash payment

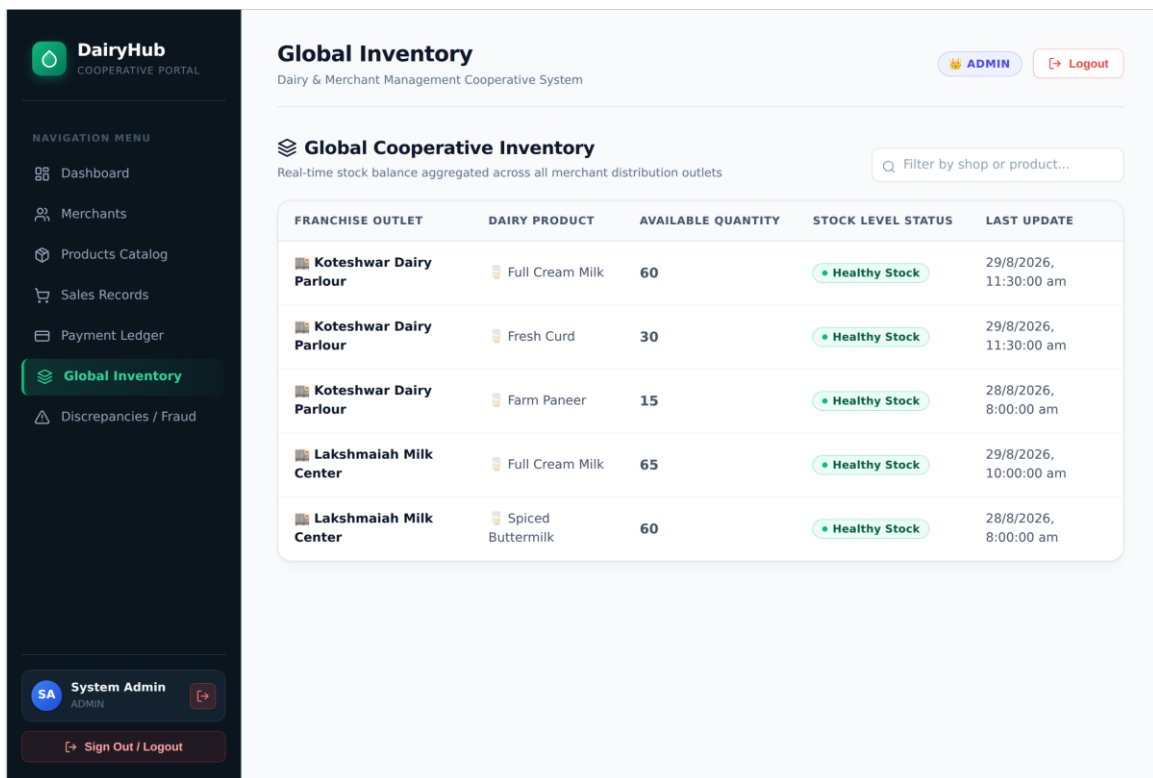


Figure 7.7: Global cooperative inventory

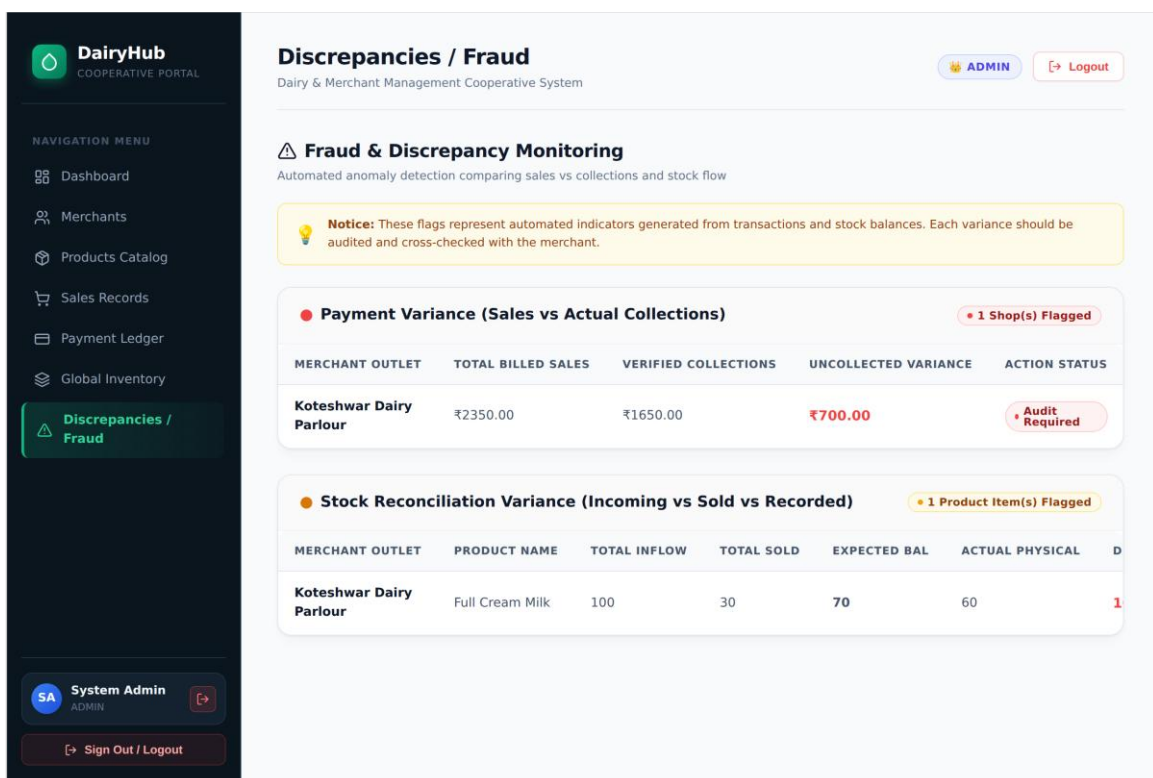


Figure 7.8: Fraud and discrepancy monitoring

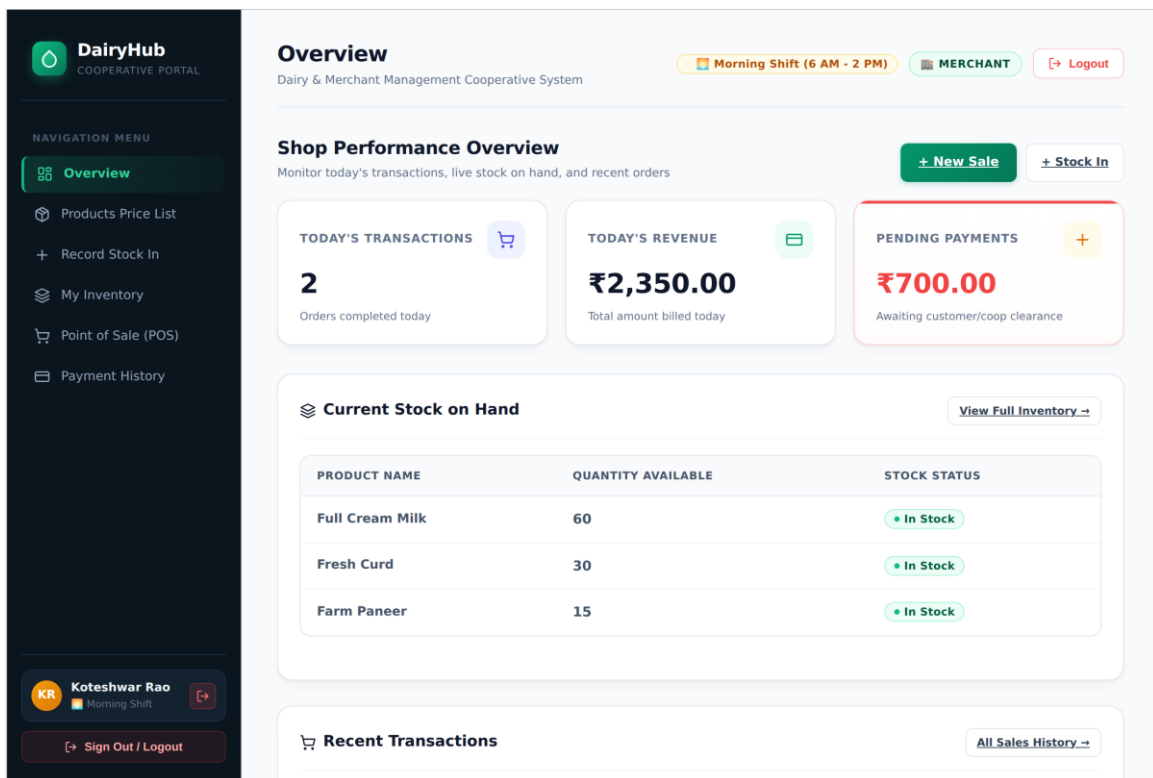


Figure 7.9: Merchant shop performance overview

DairyHub
COOPERATIVE PORTAL

NAVIGATION MENU

- Overview
- Products Price List
- + Record Stock In
- My Inventory
- Point of Sale (POS)
- Payment History

Koteshwar Rao
Morning Shift

[+] Sign Out / Logout

Record Stock In

Dairy & Merchant Management Cooperative System

Morning Shift (6 AM - 2 PM) MERCHANT Logout

+ Record Incoming Stock Batch

Log new shipments received from dairy processing plants or farmers

Stock Inflow Entry

Select Product Received

Choose dairy product...

Received Quantity Cost per Unit (₹)

e.g. 50 e.g. 32.00

Inflow Receipt Date

10/02/2026

+ Record Stock Inflow

Figure 7.10: Recording incoming stock


DairyHub
COOPERATIVE PORTAL

NAVIGATION MENU

- Overview
- Products Price List
- Record Stock In
- My Inventory
- Point of Sale (POS)**
- Payment History


Koteshwar Rao
Morning Shift
[Sign Out / Logout](#)

Point of Sale (POS)

Dairy & Merchant Management Cooperative System

Morning Shift (6 AM - 2 PM)
MERCHANT
Logout

Point of Sale (POS Terminal)

Official Cashfree Gateway Checkout & Cash Counter

New Order Billing

ORDER LINE ITEMS

Full Cream Milk (Litre) — ₹55.00	10	55	X
Farm Paneer (Kg) — ₹320.00	2	320	X

+ Add Another Product

Tax Amount (₹)

Discount (₹)

0

0

Select Payment Mode:


Cashfree UPI (Online)
Official Cashfree Gateway Modal


Cash Payment
Physical Cash Counter

CASHFREE PAYMENTS
Official Cashfree Payment Gateway
Gateway Active

TOTAL PAYABLE AMOUNT

₹1190.00

100% Secure Checkout

Accepts: UPI (GPay, PhonePe, Paytm), Debit/Credit Cards, NetBanking


Pay ₹1190.00 with Cashfree

Figure 7.11: Point of sale with Cashfree UPI payment


DairyHub
COOPERATIVE PORTAL

NAVIGATION MENU

Overview

Products Price List

Record Stock In

My Inventory

Point of Sale (POS)

Payment History

Koteshwar Rao
 Morning Shift

Sign Out / Logout

Point of Sale (POS)
Dairy & Merchant Management Cooperative System

Morning Shift (6 AM - 2 PM)

MERCHANT

Logout

Point of Sale (POS Terminal)
Official Cashfree Gateway Checkout & Cash Counter

New Order Billing

ORDER LINE ITEMS

Full Cream Milk (Litre) — ₹55.00	10	55	
Farm Paneer (Kg) — ₹320.00	2	320	

+ Add Another Product

Tax Amount (₹)

0

Discount (₹)

0

Select Payment Mode:

Cashfree UPI (Online)
Official Cashfree Gateway Modal

Cash Payment
Physical Cash Counter

Cash Received from Customer (₹)

e.g. 1190

CHANGE TO RETURN

₹0.00

Subtotal: **₹1190.00** | Tax: **₹0.00** | Discount: **-₹0.00**

Grand Total: **₹1190.00**

Complete Cash Sale

My Recent Sales Log

Figure 7.12: Point of sale with cash payment and change calculation

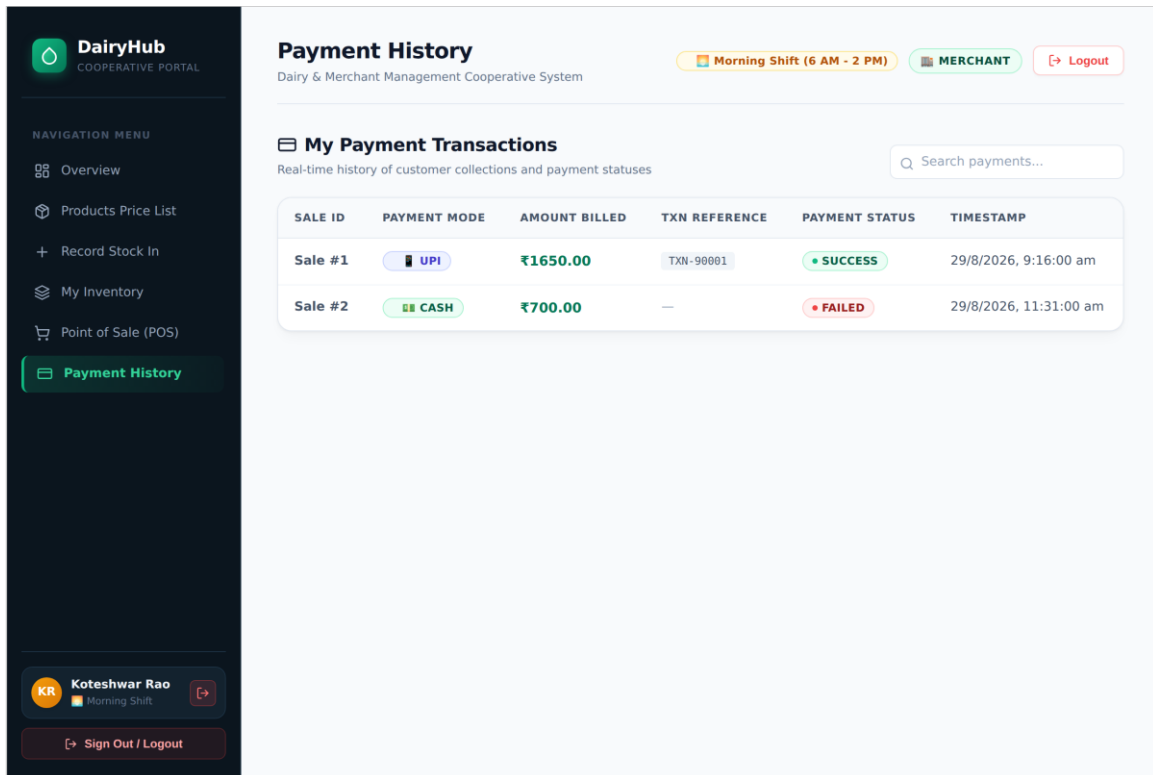


Figure 7.13: Merchant payment history

7.5 Demonstration of Database Concepts

Atomic transactions

When a sale is recorded, the invoice, sale, sale items and payment are inserted on one connection between BEGIN and COMMIT. If, for example, the payment insert fails because a transaction reference is duplicated (it is declared UNIQUE), the whole transaction is rolled back: no invoice or sale is left behind, and the saga releases the reserved stock.

Row-level locking and isolation

If two terminals try to sell the last 5 litres of milk at the same moment, the first reservation locks the inventory row with SELECT ... FOR UPDATE and deducts the quantity. The second reservation waits for the lock and, once it is released, reads the updated quantity of 0 and is rejected with an insufficient-inventory error. This demonstrates how locking prevents the lost-update anomaly.

Integrity constraints

Listing 7.4: Constraint violations rejected by MySQL

```
-- negative price violates CHECK (selling_price >= 0)
INSERT INTO products (product_name, category, unit, selling_price)
VALUES ('Test Milk', 'Milk', 'Litre', -5);
-- ERROR 3819 (HY000): Check constraint 'products_chk_1' is violated.

-- duplicate inventory row violates UNIQUE (merchant_id, product_id)
INSERT INTO inventory (merchant_id, product_id, quantity_available) VALUES (1, 1, 10);
-- ERROR 1062 (23000): Duplicate entry '1-1' for key 'inventory.uq_inventory'

-- sale for a non-existent merchant violates the foreign key
INSERT INTO sales (merchant_id, subtotal, total_amount) VALUES (99, 100, 100);
-- ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails
```

Views for fraud detection

The seed data contains two deliberate anomalies. Koteswar Dairy Parlour received 100 litres of Full Cream Milk and sold 30 litres, so 70 litres are expected, but inventory shows 60; `vw_stock_discrepancy` reports a discrepancy of 10. The same outlet billed ₹2,350 across two sales, but only ₹1,650 was paid successfully because the ₹700 cash payment is marked FAILED; `vw_payment_discrepancy` reports ₹700.

Listing 7.5: Querying the discrepancy views

```
SELECT merchant_id, product_name, expected_stock, actual_stock, discrepancy
FROM vw_stock_discrepancy WHERE discrepancy <> 0;
-- 1 | Full Cream Milk | 70.00 | 60.00 | 10.00

SELECT merchant_id, total_sales, total_successful_payments, discrepancy
FROM vw_payment_discrepancy WHERE discrepancy <> 0;
-- 1 | 2350.00 | 1650.00 | 700.00
```

Saga compensation

If the payment for a UPI sale cannot be verified, the saga does not write any sale rows. It calls the inventory service to release the reservation, which returns the reserved quantity to inventory and marks the reservation RELEASED, and records the saga as COMPENSATED with the reason in `error_log`. The integration suites in the repository exercise these paths, including controlled service failures and restarts.

CHAPTER -8 CONCLUSION

8.1 Summary of the Project

The Dairy & Milk Cooperative Management System digitises the merchant side of a dairy cooperative. Merchants record incoming stock, bill customers through a point-of-sale screen and accept cash or UPI payments, while administrators manage merchants and products and monitor revenue and anomalies across all outlets from a live dashboard.

Technically, the project combines a normalised MySQL schema with integrity constraints, indexes and analytical views; a React frontend; and a Node.js backend organised as seven microservices behind an API gateway. Consistency of the central sale workflow across services is maintained with an orchestrated saga that uses row-level locking, time-limited reservations, ACID local transactions and compensating actions.

8.2 Objectives Achieved

Table 8.1: Objectives and their status

Objective	Status	How it was achieved
Normalised relational database	Achieved	Ten 3NF tables with primary, foreign, unique and check constraints
Atomic multi-item sales	Achieved	Invoice, sale, items and payment written in one InnoDB transaction
Prevent overselling	Achieved	SELECT ... FOR UPDATE with TTL-based reservations
Cash and UPI payments	Achieved	Cash at counter; Cashfree orders, verification and refunds (sandbox)
Automatic discrepancy detection	Achieved	vw_stock_discrepancy and vw_payment_discrepancy views
Role-based dashboards	Achieved	Separate Admin/Owner and Merchant interfaces with JWT role checks
Distributed backend	Achieved	Seven services behind an API gateway, migrated from a monolith
Farmer milk collection and Fat/SNF rates	Future scope	Product quality records are in place; farmer collection was not implemented

8.3 Results Achieved

- A working web application with thirteen screens covering administration, merchant operations and reporting.
- A sale workflow that keeps stock, sales and payments consistent even when a service or the payment step fails, with automatic recovery of interrupted sales.

- Server-side pricing and validation, so a manipulated client cannot change the billed amount.
- Fraud indicators that correctly identify the two seeded anomalies: a ₹700 payment shortfall and a 10-unit stock shortfall.
- Integration test suites covering saga recovery and compensation, the reporting service, gateway cutover, controlled failure and restart, and the final system validation.

The current version also has known limitations, which are recorded here for completeness. Payments run against the Cashfree sandbox only; the CARD and ONLINE methods are stored in the ledger but are not processed through a gateway; all services share one database rather than owning separate databases; and authentication is checked inside each service rather than centrally at the gateway. These points are natural next steps for a production deployment.

8.4 Skills and Learning Outcomes

- **Database design:** translating requirements into an ER model and a normalised schema with appropriate constraints.
- **Advanced SQL:** writing joins, aggregate subqueries and views, and using indexes to support common queries.
- **Transaction management:** applying ACID transactions, isolation and row-level locking to a real concurrency problem.
- **Distributed systems:** decomposing a monolith into services, routing through a gateway, and keeping data consistent with the Saga pattern and compensating transactions.
- **Full-stack development:** building a React single-page application that consumes a REST API with JWT authentication.
- **Third-party integration:** integrating a payment gateway's order, verification and refund APIs.
- **Teamwork and tooling:** dividing frontend, backend and database work across the team, using Git and GitHub, and testing APIs with Postman and scripted integration tests.

CHAPTER -9 REFERENCES

9.1 Books

1. A. Silberschatz, H. F. Korth and S. Sudarshan, Database System Concepts, 7th ed. New York: McGraw-Hill, 2019.
2. R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed. Boston: Pearson, 2016.
3. J. Gray and A. Reuter, Transaction Processing: Concepts and Techniques. San Francisco: Morgan Kaufmann, 1992.
4. C. Richardson, Microservices Patterns: With Examples in Java. Shelter Island, NY: Manning, 2018.
5. S. Newman, Building Microservices: Designing Fine-Grained Systems, 2nd ed. Sebastopol, CA: O'Reilly Media, 2021.

9.2 Research Papers

1. E. F. Codd, "A relational model of data for large shared data banks," Communications of the ACM, vol. 13, no. 6, pp. 377–387, 1970.
2. H. Garcia-Molina and K. Salem, "Sagas," in Proc. ACM SIGMOD International Conference on Management of Data, 1987, pp. 249–259.
3. M. Jones, J. Bradley and N. Sakimura, "JSON Web Token (JWT)," IETF RFC 7519, May 2015.

9.3 MySQL / Node.js Documentation

1. Oracle Corporation, MySQL 8.0 Reference Manual. <https://dev.mysql.com/doc/refman/8.0/en/>
2. OpenJS Foundation, Node.js Documentation. <https://nodejs.org/docs/latest/api/>
3. Express.js Documentation. <https://expressjs.com/>
4. mysql2 – MySQL client for Node.js. <https://sidorares.github.io/node-mysql2/docs>

9.4 Online Resources

1. React Documentation. <https://react.dev/>

2. Vite Documentation. <https://vitejs.dev/>
3. Cashfree Payments Developer Documentation. <https://docs.cashfree.com/>
4. M. Fowler and J. Lewis, “Microservices,” 2014.
<https://martinfowler.com/articles/microservices.html>
5. Project source code repository. https://github.com/premsaisurisetty-a11y/DBSE-DBD_Project